

CSCE 605 COMPILER DESIGN · MODULE 04 · LECTURE NOTES

Parsing II: Bottom-Up

LR parsing, and what a conflict is telling you.

MODULE OUTCOMES

Explain shift-reduce parsing and the LR item construction.

Distinguish LR(0), SLR, LALR, and LR(1).

Diagnose shift-reduce and reduce-reduce conflicts.

Explain the dangling-else problem and its resolutions.

Choose between LR, LL, and GLR with reasons.

1 Shift-reduce parsing

Two actions on a stack:

SHIFT -- push the next input token

REDUCE -- the stack top matches a production's RHS; replace it with the LHS

Parsing 1 + 2 * 3 with $E \rightarrow E+T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{num}$

STACK	INPUT	ACTION
(empty)	1 + 2*3	shift
1	+ 2*3	reduce $F \rightarrow \text{num}$, $T \rightarrow F$, $E \rightarrow T$
E	+ 2*3	shift
E +	2*3	shift
E + 2	*3	reduce $F \rightarrow \text{num}$, $T \rightarrow F$
E + T	*3	SHIFT <-- not reduce $E \rightarrow E+T$. THIS is the whole decision.
E + T *	3	shift
E + T * 3	(end)	reduce $F \rightarrow \text{num}$, $T \rightarrow T * F$
E + T	(end)	reduce $E \rightarrow E+T$
E	(end)	ACCEPT

The tree is built bottom-up, leaves first — the reverse of recursive descent — and **everything difficult about LR parsing is contained in that one choice: shift, or reduce?** The item construction of §2 exists entirely to answer it mechanically. Walking this trace by hand once is the thing that makes the rest of the module make sense.

LR HANDLES LEFT RECURSION, WHICH IS THE POINT

Top-down parsing cannot handle left recursion at all (Module 03 §1) — `parse_E` calls `parse_E` having consumed nothing, and the parser loops forever.

Bottom-up parsing actively prefers it. The production $E \rightarrow E + T$ is reduced only once the entire left operand is already on the stack, so left recursion costs nothing and requires no transformation.

So the natural, directly left-associative grammar works as written, with no rewriting, no auxiliary nonterminals, and no re-association while building the tree.

And the LR grammars strictly contain the LL grammars. Every LL(1) grammar is LR(1), and the converse fails — there are grammars no amount of left-factoring makes LL. **LR accepts more languages with less grammar surgery**, which is the whole technical argument for it, and it is a good one. §4 explains why the industry nevertheless went the other way.

2 The item construction

AN ITEM IS A PRODUCTION WITH A POSITION MARKER

An item is a production with a dot marking how far through it we are. $E \rightarrow E \cdot + T$ means 'we have recognised an E and we expect to see $+ T$ next'.

A parser state is a set of items — all the productions we might currently be partway through, since the parser does not yet know which one it is in.

Closure completes a state: whenever the dot immediately precedes a nonterminal, add every production for that nonterminal with the dot at the beginning, repeating until nothing new appears. This is the same idea as epsilon-closure in Module 02.

Goto moves the dot across a grammar symbol, producing the successor state. **The resulting collection of states is a DFA over grammar symbols** — constructed by exactly the subset construction of Module 02, applied to items rather than to characters. **That the same construction appears twice is not a coincidence;** both are determining a nondeterministic recognition process.

Variant	Lookahead used	State count	Character
LR(0)	None at all.	Fewest.	Too weak for any realistic language — it cannot even handle a grammar where a nonterminal is sometimes followed by different tokens. Pedagogically useful as the base construction.
SLR(1)	The global FOLLOW set of the nonterminal being reduced.	Same as LR(0).	Simple, and rejects many useful grammars because the global FOLLOW set is coarser than the context actually permits.

Variant	Lookahead used	State count	Character
LALR(1)	LR(1) lookaheads, merged across states with identical cores.	Same as LR(0) — which is the entire point.	What yacc, bison, and most generators produce, and the practical choice. The merge is what keeps the tables small enough to have been shippable in 1975.
LR(1)	Exact lookahead, computed per state.	Many more — often an order of magnitude.	The most powerful of the deterministic family , with tables that were impractical historically and are fine now. Menhir produces these.
GLR	Forks the parse on every conflict and runs all possibilities.	Varies dynamically.	Handles genuinely ambiguous grammars and returns a parse <i>forest</i> rather than a tree, leaving disambiguation to a later phase.

LALR's merge is worth understanding, because it causes confusing conflicts. Two LR(1) states with the same items but different lookaheads are merged into one, which keeps the table small — but the merged lookahead sets are unions, and **that union can create a reduce-reduce conflict that neither original state had.** The resulting bison error describes a problem that does not exist in your grammar as LR(1), which is deeply unhelpful if you do not know the merge is happening.

3 Conflicts

A CONFLICT MEANS THE GRAMMAR IS AMBIGUOUS AT THAT POINT

A **shift-reduce conflict** means that in some state, with some lookahead, the parser could legitimately either shift or reduce and the lookahead does not settle it. **This is usually a real ambiguity in the grammar**, and precedence and associativity declarations normally resolve it cleanly.

A **reduce-reduce conflict** means two different productions could both be reduced at the same point. **This is almost always a genuine grammar bug** — two constructs that the grammar has made indistinguishable — and precedence declarations will not help.

Generators resolve shift-reduce conflicts by shifting, by default, and usually silently. That default happens to be what you want surprisingly often, which is how the dangling-else case below gets accidentally right in a great many grammars.

But never ship with unresolved conflicts. The default is a guess made by the tool, and **the grammar you have is then not the grammar you think you have** — the language your parser accepts has been decided by a conflict-resolution rule rather than by you. Treat conflict count as a build error, not a warning.

```
S -> if E then S
    | if E then S else S
    | other

"if a then if b then x else y"  -- which `if` owns the `else`?

SHIFT  -> if a then (if b then x else y)
REDUCE -> if a then (if b then x) else y

RESOLUTIONS:
1. SHIFT (the default): else binds to the NEAREST if.
   C, Java, C# -- and it is what everyone expects.
2. Rewrite into "matched" and "unmatched" statements.
   Unambiguous, and much harder to read.
3. REQUIRE braces. Rust, Go, Swift: the ambiguity cannot arise.
```

Option 3 is a language-design fix for a parsing problem, and it is the better place to solve it. Every language designed after this ambiguity was thoroughly understood chose to require braces — which eliminates not only the parsing difficulty but also the whole class of reader confusion and the goto fail family of bugs. **A construct that is hard to parse is frequently also hard for a human to read**, so parser difficulty is worth treating as a design signal rather than purely as an implementation obstacle.

4 Choosing a parsing technology

Use	When	Who does
Hand-written recursive descent plus Pratt	You control the language, and diagnostics and IDE support matter.	GCC (since 2004), Clang, Rust, Go, V8, TypeScript.

Use	When	Who does
LALR generator (bison, yacc)	The grammar is given, large, and stable, and development speed matters more than error messages.	Many DSLs and configuration languages; most older compilers.
LR(1) generator (Menhir)	You hit LALR merge conflicts you cannot resolve, and want the stronger guarantee.	OCaml tooling; research compilers.
GLR	The language is genuinely ambiguous and disambiguation needs semantic information.	C++ in some analysis tools; natural language parsing.
PEG / packrat	You want to write the grammar directly as composable code.	Ordered choice silently hides ambiguity rather than reporting it, which is a genuine drawback — you get a parse, not a warning.

WHY PRODUCTION COMPILERS ABANDONED GENERATORS

GCC used bison until 2004 and then replaced it with a hand-written recursive-descent parser. Clang never used one. The textbook default lost, and it is worth knowing why.

Diagnostics were the main reason. A generated parser knows it is in state 247 with a particular lookahead; **it does not know that you forgot a semicolon on the previous line**, and recovering that intent from the automaton's state is extremely awkward. Recursive descent knows it is in `parse_if` and can say so.

Error recovery is the second. Panic-mode recovery (Module 03 §4) is natural when the parser is a call stack of functions that each know their own FOLLOW set, and clumsy when it is a table-driven automaton whose stack holds integers.

And modern IDEs need incremental, error-tolerant parsing of code that is *almost always* briefly invalid while it is being typed, re-parsed on every keystroke, with a usable tree produced anyway. **That requirement simply did not exist when yacc was designed in the 1970s**, and it is now the dominant one — which is the real story here: the technology did not get worse, the requirements changed underneath it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS143 — bottom-up parsing lectures (free)	The item construction and the LR family of §2, worked through carefully.
Aho, Lam, Sethi & Ullman — <i>Compilers (the Dragon Book)</i> , chapter 4	The definitive treatment of LR parsing. Library copy; this chapter is the book's strongest.
Bison manual — conflict reporting and resolution (free)	How to read a conflict report , which is the practically important skill of §3.

Resource	What it covers for this module
Tree-sitter — documentation and design (free)	Incremental, error-tolerant GLR parsing for editors — the §4 requirement, taken seriously and generated rather than hand-written.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Trace a shift-reduce parse by hand** for an expression with both operators, writing out the stack at every step.
- 2 Build the LR(0) item sets for a small grammar.
- 3 Compute SLR lookaheads and find a grammar SLR rejects but LALR accepts.
- 4 Run bison on an ambiguous grammar and read the conflict report.
- 5 **Resolve a shift-reduce conflict with precedence declarations** and confirm the conflict count reaches zero.
- 6 **Construct a reduce-reduce conflict** deliberately and explain the grammar bug behind it.
- 7 Implement the dangling-else grammar both ways and compare the parse trees.
- 8 **Find an LALR conflict caused by state merging** that LR(1) does not have.
- 9 Generate a parser for your project grammar and compare it against your hand-written one on error quality.
- 10 Parse a file with a deliberate error in both and compare the messages.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe shift-reduce parsing and name the one hard decision.
- 2 Why does LR handle left recursion when LL cannot?
- 3 What is an item, a state, closure, and goto?
- 4 Compare LR(0), SLR, LALR, LR(1), and GLR.
- 5 What does LALR's merge buy and what does it cost?
- 6 Distinguish shift-reduce from reduce-reduce conflicts.
- 7 State the dangling-else problem and three resolutions.
- 8 Give three reasons production compilers hand-write parsers.